



TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE HERMOSILLO

Programación Web

Ingeniería en Sistemas Computacionales – 6to semestre

GRUPO S6B

Sistema de gestión y venta de abarrotes

*Alumnos: Medina Llanez Brian Adib
Celaya Campuzano Jesus Guadalupe
Rendon Villarreal America Lizeth*

Docente: Moroyoqui Ibarra Jonathan Adrián

Fecha de entrega: 27 de mayo de 2026

INTRODUCCION

Descripción general del sistema desarrollado

El sistema desarrollado es una aplicación web para la gestión integral de una tienda de abarrotes, construida con ASP.NET Core Web API en el backend y React con Vite en el frontend. La plataforma permite administrar el catálogo de productos, controlar el inventario, registrar ventas y gestionar la información de proveedores y usuarios del sistema. La base de datos fue diseñada e implementada con SQL Server utilizando Entity Framework Core bajo el enfoque Code First, lo que permitió generar y versionar la estructura de la base de datos directamente desde el código mediante migraciones. El sistema también cuenta con un módulo de autenticación basado en JWT, el cual restringe el acceso a las funcionalidades según el usuario registrado.

Problema que resuelve o necesidad que cubre

Las tiendas de abarrotes de pequeño y mediano tamaño típicamente llevan el control de su inventario y sus ventas de forma manual, ya sea en papel o en hojas de cálculo sin ninguna estructura formal. Esto genera problemas como pérdidas de información, errores en el conteo de existencias, dificultad para identificar productos agotados o con bajo stock, y falta de un registro confiable de las ventas realizadas. Además, la ausencia de control de acceso permite que cualquier persona pueda modificar información crítica del negocio. Este sistema surge como solución a esas necesidades, ofreciendo una plataforma centralizada que digitaliza y organiza los procesos de inventario y punto de venta, con acceso controlado mediante autenticación de usuarios.

Objetivo general del proyecto

Desarrollar una aplicación web completa (backend + frontend) para la gestión de una tienda de abarrotes que permita administrar el catálogo de productos, el inventario y las ventas, con autenticación de usuarios y diferenciación de roles, cumpliendo con los requisitos técnicos establecidos (ASP.NET Core 8, React, Entity Framework Code First, migraciones, CRUD de al menos tres catálogos, dos módulos funcionales, y documentación profesional).

DOCUMENTACION TÉCNICA

Tecnologías utilizadas:

Backend (.NET):

.NET SDK: 8.0 (o 10.0.103, compatible con net8.0)

Librerías NuGet principales:

- Microsoft.EntityFrameworkCore.SqlServer 8.0.0
- Microsoft.EntityFrameworkCore.Tools 8.0.0
- Microsoft.AspNetCore.Authentication.JwtBearer 8.0.0
- BCrypt.Net-Next 4.2.0
- Swashbuckle.AspNetCore (Swagger) – versión incluida por defecto en la plantilla

Frontend (React):

React: 18.3.1

Librerías npm principales:

- react-router-dom 6.23.1
- axios 1.6.0 (o la última instalada)
- vite 5.2.0

Base de datos:

SQL Server: Microsoft SQL Server 2022 (RTM) - 16.0.1000.6 (X64) Oct 8 2022 05:58:25 Copyright (C) 2022 Microsoft Corporation Developer Edition (64-bit) on Windows 10 Pro 10.0 <X64> (Build 26200:) (Hypervisor)

Arquitectura:

- Backend sigue el patrón MVC (sin vistas, solo API).
- Frontend está completamente separado dentro de la subcarpeta vista/, con su propio package.json y estructura de componentes (src/pages, src/components, src/context, src/api.js).

Descripción de la estructura del proyecto

El sistema emplea una arquitectura Cliente-Servidor, separando estrictamente la interfaz de usuario de la lógica de negocio para garantizar escalabilidad y facilitar su mantenimiento.

- Capa de Presentación (Frontend): Desarrollada en React bajo un diseño de componentes modulares. Se divide en vistas principales, elementos de interfaz reutilizables, y la gestión del estado global y el enrutamiento protegido (mediante react-router-dom).

- Capa de Lógica y Datos (Backend): Construida en .NET, centraliza las operaciones del sistema. Se estructura en Controladores (puntos de entrada de la API), Modelos (representación de las tablas) y el contexto de datos (Entity Framework Core) para interactuar de forma segura con SQL Server.

Explicación de la comunicación entre Front-end y Back-end

- Ambas capas se comunican de forma asíncrona mediante una API RESTful a través del protocolo HTTP, operando bajo las siguientes directrices:
- Intercambio de Datos: Toda la información enviada y recibida entre cliente y servidor se procesa en formato ligero JSON.
- Peticiones HTTP: El frontend utiliza la librería axios para ejecutar las operaciones de la base de datos mediante los métodos estándar: GET (leer), POST (crear), PUT (actualizar) y DELETE (eliminar).
- Seguridad: La comunicación está blindada mediante JWT (JSON Web Tokens). Tras iniciar sesión, el frontend almacena un token criptográfico y lo adjunta en los encabezados de cada petición. El backend valida esta firma antes de procesar cualquier transacción o conceder acceso a los datos.

Resumiendo, la comunicación entre ambas capas se realiza de forma asíncrona a través de una API RESTful. El cliente (frontend) utiliza la librería axios para intercambiar datos en formato JSON mediante peticiones HTTP estándar (GET, POST, PUT, DELETE). Todo este flujo está protegido por tokens JWT, los cuales el cliente adjunta en cada solicitud para que el servidor (backend) valide la identidad y autorice las transacciones. De ser correcta, procesa la solicitud; de lo contrario, retorna un error 401. Este mecanismo asegura que solo usuarios autenticados puedan acceder a los recursos protegidos.

Base de datos:

El sistema que creamos utiliza una base de datos relacional teniendo en cuenta el enfoque Code-First de Entity Framework Core.

Descripción de cada entidad y sus atributos principales

Entidad	Atributos	Descripción
Usuario	Id (PK), Username, PasswordHash, Role	Representa a los operadores del sistema. El campo Role puede ser Admin, Vendedor o Almacenista y define los permisos.
Categoría	Id (PK), Nombre	Clasificación de los productos (por ejemplo, Bebidas, Snacks, Dulces, etc).
Proveedor	Id (PK), Nombre	Empresa o persona que suministra los productos.
Producto	Id (PK), Nombre, Precio, Stock, CategoriaId (FK), ProveedorId (FK)	Artículo disponible para la venta. El campo Stock controla el inventario.
Venta	Id (PK), Fecha, Total, UsuarioId (FK)	Encabezado de cada transacción de caja. Total es la suma de los subtotales de los detalles.
DetalleVenta	Id (PK), VentaId (FK), ProductoId (FK), Cantidad, PrecioUnitario	Líneas de cada venta. PrecioUnitario se guarda en el momento de la venta para mantener el histórico, aunque el precio del producto cambie.

Nota de diseño: El campo Subtotal es una propiedad calculada en tiempo de ejecución ($\text{Cantidad} * \text{PrecioUnitario}$), evitando redundancia en el almacenamiento.

Relaciones entre tablas

Producto → Categoría: Relación muchos a uno. Un Producto pertenece a una Categoría (mediante CategoríaId). Una Categoría puede tener muchos Productos.

Producto → Proveedor: Relación muchos a uno. Un Producto es suministrado por un Proveedor. Un Proveedor puede suministrar muchos Productos.

Venta → Usuario: Relación muchos a uno. Una Venta es registrada por un Usuario. Un Usuario puede registrar muchas Ventas.

DetalleVenta → Venta: Relación muchos a uno. Cada DetalleVenta pertenece a una Venta. Una Venta puede tener muchos DetallesVenta.

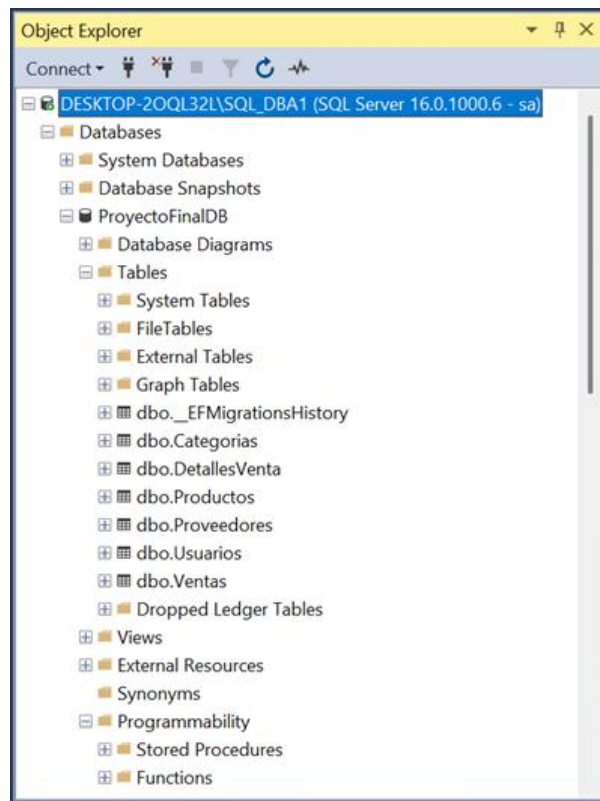
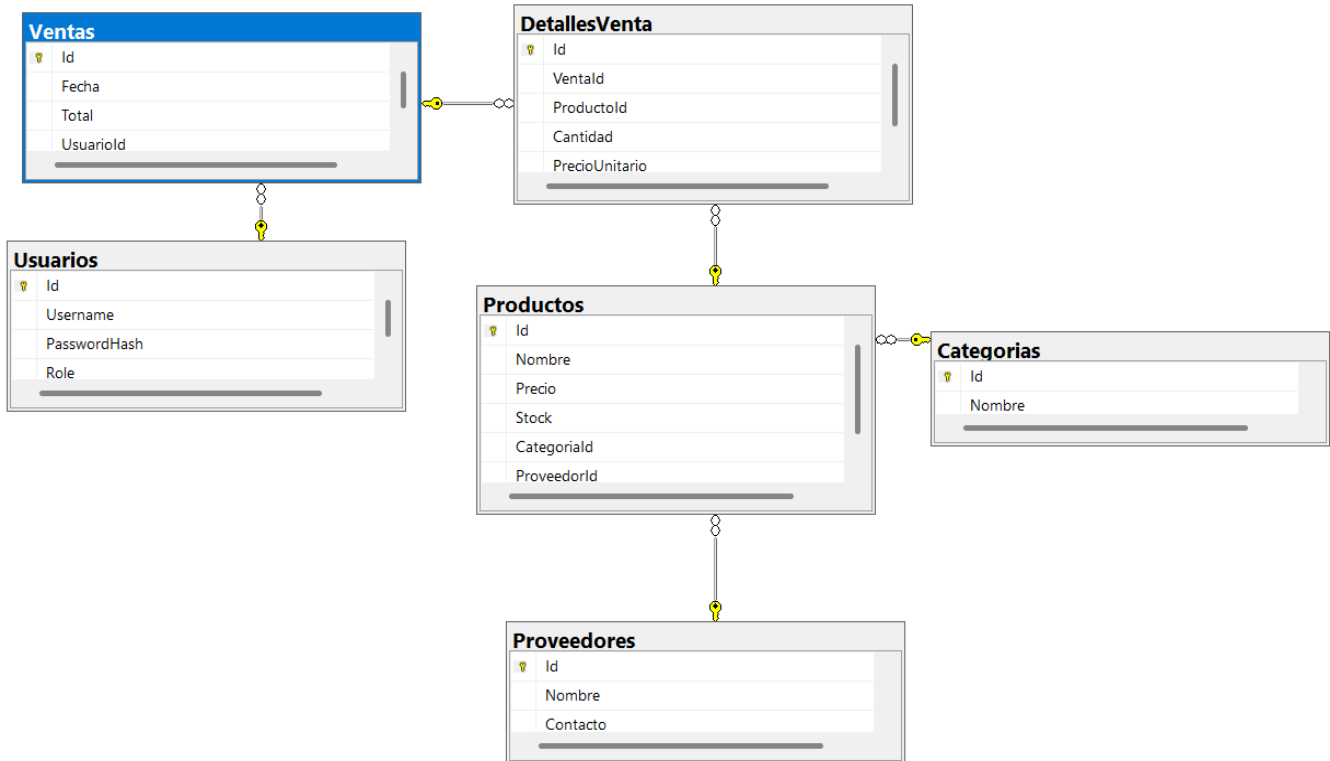
DetalleVenta → Producto: Relación muchos a uno. Cada DetalleVenta referencia al Producto vendido. Un Producto puede aparecer en muchos DetallesVenta.

Las conexiones lógicas se establecen mediante restricciones de integridad referencial (Llaves Foráneas), configuradas de la siguiente manera:

Claves foráneas (FK)

- Producto.CategoríaId → Categoría.Id
- Producto.ProveedorId → Proveedor.Id
- Venta.UsuarioId → Usuario.Id
- DetalleVenta.VentaId → Venta.Id
- DetalleVenta.ProductoId → Producto.Id

*Comportamiento en cascada. Las migraciones de Entity Framework Core están configuradas para eliminar en cascada las dependencias fuertes (por ejemplo, al eliminar una Venta, se eliminan automáticamente todos sus DetallesVenta). Para las relaciones con Producto, se evita la eliminación si existen registros de ventas asociados (integridad referencial).



API:

Estos son los endpoints de nuestra la API RESTful. Todos los endpoints que modifican datos requieren autenticación mediante token JWT. Algunos también requieren roles específicos (Admin, Vendedor o Almacenista).

Listado de endpoints desarrollados con método HTTP, ruta y descripción

- Módulo de Autenticación (AuthController) - Gestiona el registro de personal y la emisión de tokens para control de acceso.

POST /api/auth/registrar --> Registrar un nuevo usuario (Admin, Vendedor, Almacenista)

POST /api/auth/login --> Iniciar sesión de usuario y obtener el token JWT

- Módulo de Productos (ProductosController) - Controla el catálogo y las existencias físicas de las mercancías del negocio.

GET /api/productos --> Obtener la lista de todos los productos en existencia

GET /api/productos/{id} --> Obtener los detalles de un producto específico mediante su ID

POST /api/productos --> Crear e integrar un nuevo producto al catálogo

PUT /api/productos/{id} --> Actualizar el nombre, precio, stock o datos de un producto

DELETE /api/productos/{id} --> Eliminar o dar de baja un producto del sistema

- Módulo de Categorías (CategoriasController) - Administra las clasificaciones comerciales para la organización del inventario.

GET /api/categorias --> Obtener la lista de todas las categorías registradas

GET /api/categorias/{id} --> Obtener una categoría por su ID

POST /api/categorias --> Crear una nueva categoría de productos

PUT /api/categorias/{id} --> Modificar el nombre de una categoría existente

DELETE /api/categorias/{id} --> Eliminar una categoría del sistema

- Módulo de Proveedores (ProveedoresController) - Administra el directorio de los contactos comerciales que surten la tienda.

GET /api/proveedores --> Obtener la lista de todos los proveedores registrados

GET /api/proveedores/{id} --> Obtener los datos de contacto de un proveedor por su ID

POST /api/proveedores --> Registrar un nuevo proveedor en la base de datos

PUT /api/proveedores/{id} --> Actualizar el nombre o contacto de un proveedor existente

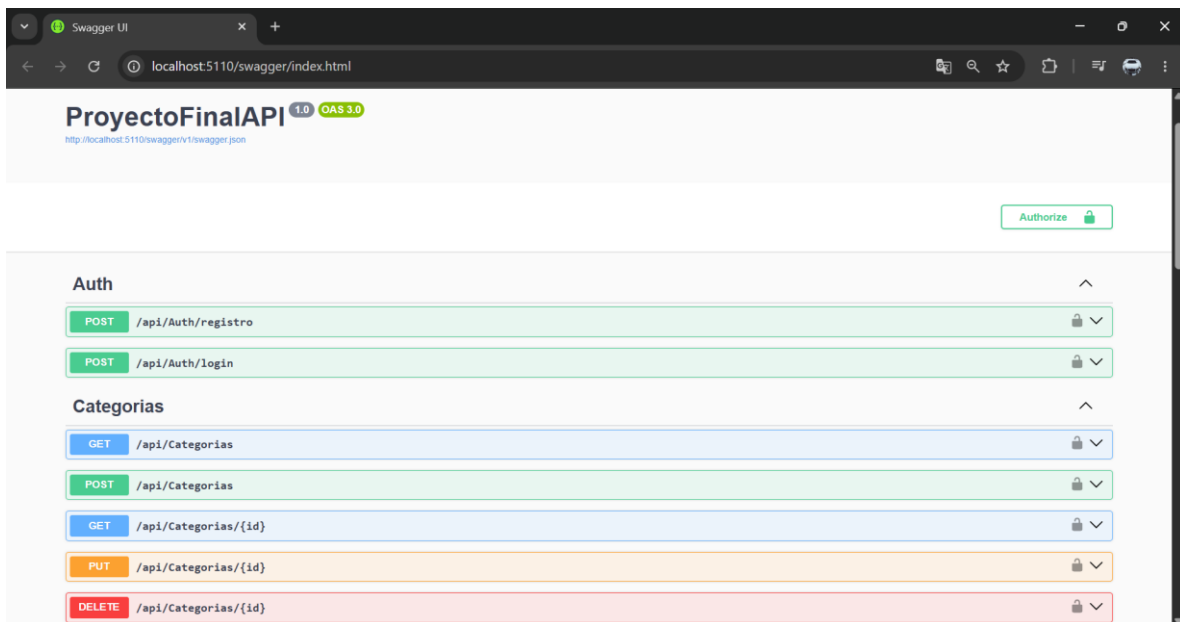
DELETE /api/proveedores/{id} --> Eliminar un proveedor del registro

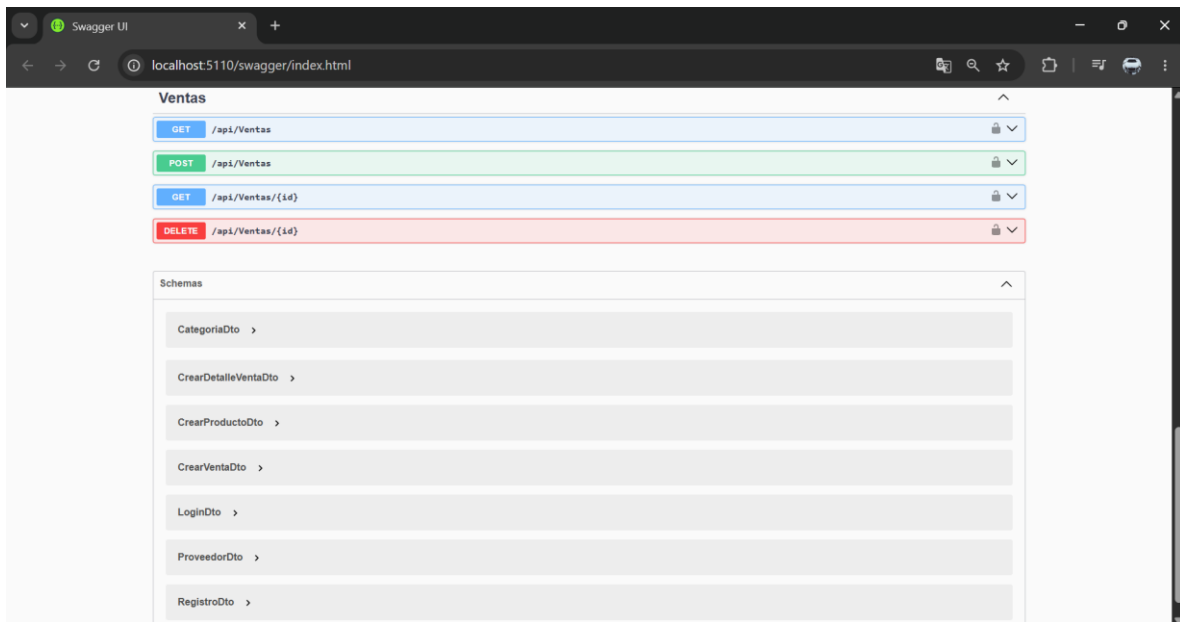
- Módulo de Ventas (VentasController) - Procesa las transacciones de la caja del punto de venta y resguarda el histórico de folios.

GET /api/ventas --> Obtener el historial de todas las ventas globales

GET /api/ventas/{id} --> Obtener el desglose y artículos de una venta por su ID de folio

POST /api/ventas --> Registrar una nueva venta (procesa el carrito y descuenta stock)





DISTRIBUCION DEL TRABAJO

Integrante	Contribuciones en el Backend	Contribuciones en el Frontend
Brian Medina - (brianadib)	- Configuración inicial del proyecto .NET (estructura, paquetes, CORS). - Creación de todos los modelos de datos (Usuario, Producto, Categoria, Proveedor, Venta, DetalleVenta). - Implementación del DataSeeder con carga desde JSON. - Desarrollo de controladores completos. - Implementación de DTOs. - Configuración de autenticación JWT y protección de endpoints. - Corrección de problemas de serialización y ciclos. - Configuración de migraciones y aplicación de base de datos.	- Inicialización del proyecto React con Vite. - Configuración del proxy y del cliente Axios (api.js). - Desarrollo de las vistas: Dashboard, Productos, Proveedores, Categorías, Ventas. - Implementación de formularios y modales para CRUD. - Integración con la API y manejo de respuestas (extracción de \$values). - Mejoras de estilos (CSS personalizado).
Jesús Celaya - (celaya03)	Creación de modelos iniciales y configuración	Desarrollo de la vista Login y Registro.

	de Entity Framework (AppDbContext). • Desarrollo de Controladores CRUD (Productos, Categorías, Proveedores). • Implementación de seguridad (AuthController y tokens JWT). • Lógica transaccional de ventas y creación de DTOs.	• Ajustes de diseño y tema (colores rojos-blanco, fuentes). • Correcciones en la navegación y símbolos de la interfaz.
América Rendón - (useer-22)	• Colaboró en la validación de DTOs. • Probó los endpoints de autenticación y ventas. • Documentó la API en Swagger. • Apoyó en la configuración de CORS.	• Creación de la UI para el módulo de Categorías (incluyendo modales y validaciones). • Integración de peticiones Axios para Categorías. • Desarrollo del Punto de Venta (Carrito de compras y cálculo de totales/subtotales). • Integración de la creación de ventas con el backend.

RETROALIMENTACIÓN DEL PROYECTO

América Lizeth Rendón Villarreal

¿Qué fue lo más difícil de desarrollar y cómo lo resolviste?

Lo más difícil fue el control de versiones con Git, manejar las ramas, los merges y que todo quedara bien integrado sin conflictos no es tan sencillo al principio. Lo fui resolviendo con práctica y comunicación con el equipo para no pisarnos el trabajo.

¿Qué herramientas o recursos externos consultaste?

Principalmente la documentación oficial de Microsoft, algunos videos en YouTube y Swagger para ir probando los endpoints

¿Qué cambiarías o mejorarías si tuvieras más tiempo?

Le agregaría reportes o gráficas de ventas y mejoraría un poco el diseño del frontend, creo que visualmente se puede pulir bastante más.

¿Qué aprendiste que no sabías antes de iniciar el proyecto?

Aprendí cómo conectar un backend con un frontend de forma real, cómo funcionan las migraciones de EF Core y cómo proteger rutas con JWT, cosas que en la práctica se sienten muy diferentes a solo verlas en clase.

Celaya Campuzano Jesus Guadalupe

¿Qué fue lo más difícil de desarrollar y cómo lo resolvieron?

Lo más difícil no fue tanto el desarrollo técnico en sí, sino la coordinación del equipo al momento de trabajar con Git. En varias ocasiones se subieron cambios a ramas incorrectas o se sobrescribieron archivos que no debían modificarse, lo que generó conflictos que tuvimos que resolver manualmente revisando el historial de commits y restaurando los archivos afectados.

¿Qué herramientas o recursos externos consultaron?

Principalmente la documentación oficial de Microsoft para ASP.NET Core y Entity Framework Core, la documentación de React y React Router, y Stack Overflow para resolver dudas puntuales durante el desarrollo.

¿Qué cambiarían o mejorarían si tuvieran más tiempo?

Mejoraría la organización del equipo en cuanto al flujo de trabajo con Git, estableciendo reglas más claras desde el inicio sobre qué se puede subir a cada rama y quién es responsable de cada parte del código.

¿Qué aprendieron que no sabían antes de iniciar el proyecto?

Al estar cursando décimo semestre, la mayoría de los conceptos ya los conocía de materias anteriores, incluyendo el despliegue de aplicaciones en plataformas como Vercel. Sin embargo, el proyecto me permitió reforzar la integración completa entre un backend en .NET y un frontend en React consumiendo una API REST con autenticación JWT, trabajando de forma colaborativa en equipo con Git, algo que siempre tiene sus particularidades en la práctica.

Medina Llanez Brian Adib

¿Qué fue lo más difícil de desarrollar y cómo lo resolvieron?

Lo más complicado fue el desarrollo del backend, la implementación del DataSeeder para poblar la base de datos con datos de prueba sin que se duplicaran. Al principio, cada vez que ejecutábamos el proyecto se volvían a insertar los mismos registros, generando acumulación de productos vacíos o repetidos. El problema era que el seeder no verificaba si ya existían datos antes de insertar. Lo resolvimos añadiendo una validación al inicio del SeedAsync: `if (await context.Productos.AnyAsync()) return;`, que evita la reinserción si ya hay productos.

Por otro lado, el control de versiones con Git especialmente al trabajar con ramas, resolver conflictos de merge y coordinar Pull Requests.

¿Qué herramientas o recursos externos consultaron?

Principalmente consultamos YouTube para entender conceptos como la creación de bases de datos sin T-SQL, la configuración de autenticación JWT y el manejo de relaciones entre tablas. También recurrimos a GitHub Docs para entender el flujo con ramas y Pull Requests.

¿Qué cambiarían o mejorarían si tuvieran más tiempo?

Si contáramos con más tiempo, mejoraríamos notablemente el frontend. Aunque las vistas son funcionales, podríamos hacerlas más atractivas y alineadas con la identidad de una tienda de abarrotes.

¿Qué aprendieron que no sabían antes de iniciar el proyecto?

Aprendí el uso de JWT para asegurar la comunicación entre el frontend y el backend, y crear bases de datos sin escribir una sola línea de SQL. Antes pensaba que cualquier base de datos requería scripts en SQL, pero usando este enfoque, todo se genera automáticamente a partir de las clases. Esto nos ahorró mucho trabajo y evitó errores manuales. También mejoramos nuestro manejo de Git y trabajo colaborativo.

RETROALIMENTACIÓN DEL CURSO

América Lizeth Rendón Villarreal

¿Qué temas del curso te fueron más útiles para el proyecto?

Todo lo de Entity Framework Core y la estructura de una Web API fue lo que más se usó durante el proyecto.

¿Qué temas sentiste que necesitaban más práctica o explicación?

JWT y el consumo de la API desde React, se vieron en clase, pero en la práctica se siente que les faltó un poco más de ejercicio.

¿Qué le agregarías o quitarías al curso?

Le agregaría algún ejercicio práctico integrador antes del proyecto final, para llegar con más confianza. No le quitaría nada.

¿Cómo calificarías el curso del 1 al 10 y por qué?

Un 10, porque los temas están bien elegidos y todo lo del proyecto final sí se vio en clase, y el que no aprendió fue porque no quiso.

Celaya Campuzano Jesús Guadalupe

¿Qué temas del curso fueron más útiles para el proyecto?

De los temas expuestos al inicio del curso, los que más aportaron al proyecto fueron la Arquitectura de las aplicaciones web, ya que nos ayudó a entender cómo estructurar correctamente la comunicación entre el frontend y el backend, y las Tecnologías para el desarrollo web, que nos dio un panorama general de las herramientas que usaríamos. También fueron muy útiles los temas de Seguridad e interoperabilidad al momento de implementar la autenticación con JWT y los Patrones de diseño y estándares al organizar la estructura del proyecto con Controllers, Models, DTOs y servicios.

¿Qué temas sintieron que necesitaban más práctica o explicación?

El tema de Plataformas tecnológicas y Conceptos generales de cómputo en la nube se quedaron un poco en lo teórico y hubiera sido interesante llevarlos más a la práctica, por ejemplo, desplegando el proyecto en algún servicio en la nube como Azure o AWS.

¿Qué le agregarían o quitarían al curso?

Agregaría una práctica dedicada al trabajo colaborativo con Git en equipo y también una introducción al despliegue en la nube para completar el ciclo de desarrollo de una aplicación real.

¿Cómo calificarías el curso del 1 al 10 y por qué?

Le daría un 8. Al venir de décimo semestre con experiencia previa en desarrollo web y despliegue de aplicaciones en plataformas como Vercel, el curso no representó un reto técnico mayor, pero sí fue una buena oportunidad para consolidar conocimientos y trabajar en equipo con tecnologías del mundo real como ASP.NET Core y React.

Comentarios adicionales para el profesor:

El proyecto final es una buena actividad integradora. Sería interesante en futuras ocasiones incluir una sesión práctica de despliegue en la nube para que los alumnos vean su proyecto funcionando en un entorno real y no solo de manera local.

Medina Llanez Brian Adib

¿Qué temas del curso fueron más útiles para el proyecto?

El tema que resultó más útil fue sin duda la estructuración de una aplicación web completa, desde la separación de capas hasta la organización de carpetas. Aprender a diferenciar los controladores, los modelos, los DTOs y el contexto de base de datos me permitió construir el proyecto de forma ordenada y entender por qué cada pieza va en su lugar.

¿Qué temas sintieron que necesitaban más práctica o explicación?

Considero que el tema de autenticación y autorización fue el que más nos costó y donde sentimos que faltó mayor profundidad o ejercicios prácticos. También habría sido útil practicar más con la configuración de CORS y el manejo de errores en el frontend.

¿Qué le agregarían o quitarían al curso?

Yo quitaría la parte de diseño con CSS y tipografías porque no es el foco principal del desarrollo web. En su lugar, agregaría más contenido sobre control de versiones con Git en un entorno profesional como trabajar con ramas, resolver conflictos de merge, y utilizar Pull Requests con revisiones.

¿Cómo calificarías el curso del 1 al 10 y por qué?

Le doy un 8. La materia es buena, creo que faltó profundizar en temas clave como la creación de API RESTful desde cero y el manejo de autenticación con tokens. Además, algunos conceptos se vieron muy rápido y hubiera sido mejor con más ejemplos prácticos.

Comentarios adicionales para el profesor

La materia es muy interesante y me llevo una base sólida para seguir aprendiendo por mi cuenta.

CONCLUSIONES - Conclusión general del equipo

América Lizeth Rendón Villarreal

Conclusión general del equipo sobre el proyecto

Logramos desarrollar un sistema funcional que cubre las necesidades de una tienda de abarrotes, aplicando todo lo aprendido durante el semestre en un proyecto real.

¿Lograron el objetivo planteado al inicio?

Sí, el sistema quedó con los módulos que nos propusimos desde el inicio y cumple con lo que se pedía.

Reflexión final sobre el trabajo en equipo

Dividir bien las tareas y comunicarnos hizo que todo fluyera mejor, como mis compañeros son de semestres más adelante que yo me enseñaron muchas cosas, aunque a veces colmaba su paciencia, gracias a ellos aprendí muchas cosas a lo largo del desarrollo del proyecto, si volvería hacer equipo con ellos 10/10

Celaya Campuzano Jesús Guadalupe

Conclusión general del equipo sobre el proyecto

El desarrollo del sistema de gestión para Abarrotes Don Pepe fue una experiencia que nos permitió aplicar de manera integral los conocimientos adquiridos a lo largo del curso. Logramos construir una aplicación web completa con un backend en ASP.NET Core conectado a SQL Server mediante Entity Framework Core y un frontend en React que consume la API REST con autenticación basada en JWT y control de roles.

¿Lograron el objetivo planteado al inicio?

Sí, se logró el objetivo planteado. El sistema cuenta con las funcionalidades principales de gestión de productos, categorías, proveedores y ventas, con un control de acceso por roles que diferencia entre Administrador, Vendedor y Almacenista. La base de datos fue generada completamente desde las migraciones de EF Core y el frontend consume la API sin datos hardcodeados.

Reflexión final sobre el trabajo en equipo

El trabajo en equipo fue uno de los mayores aprendizajes del proyecto. Aunque el desarrollo técnico fue manejable, la coordinación y organización del equipo, especialmente en el manejo de ramas con Git, representó un reto real que nos enseñó la importancia de establecer flujos de trabajo claros desde el inicio de cualquier proyecto colaborativo.

Medina Llanez Brian Adib

Conclusión general del equipo sobre el proyecto

Crear el sistema no enseñó a aplicar todos lo que vimos en el semestre en un entorno real y aprender muchas cosas más. Logramos construir una aplicación web completa, con un backend que consume la API. Estoy muy contento con el DataSeeder que se implementó para cargar datos de prueba desde un archivo JSON.

¿Lograron el objetivo planteado al inicio?

Sí, el sistema quedó con los módulos y las vista que nos propusimos desde un principio. Por eso, consideramos que se alcanzaron todos los objetivos académicos y funcionales.

Reflexión final sobre el trabajo en equipo

Aprendimos muchas cosas nuevas y recordamos unas otras tantas. Pienso que me sirvió mucho para volver aprender a trabajar en equipo, que es algo muy importante en cuanto a desarrollos de software porque casi nunca se trabaja solo.